

CENTRAL BOARD OF SECONDARY EDUCATION
NEW DELHI



MINESWEEPER

Computer Science Project

By

Roll No. : 16

Student Name : LENNY ELIZABETH JACOB

Grade : XII 'A' (PCMC)



NAGARJUNA VIDYANIKETAN

#104, IVRI Road, Behind CRPF

Yelahanka, Bangalore-560064

BONAFIDE CERTIFICATE



NAGARJUNA VIDYANIKETAN

This is to certify that **Lenny Elizabeth Jacob** student of Grade 12 has pursued her work with diligence and prepared the dissertation entitled '**Minesweeper**' under my supervision. This is being submitted as partial fulfilment for the All India Senior School Certificate Examination for the academic year 2022-2023.

In Charge Teacher's signature

Principal's signature

Internal Examiner's signature

External Examiner's

ACKNOWLEDGEMENT

The success and final outcome of this required a lot of guidance and assistance from many people and I am extremely fortunate to have got this all along the completion of my project work. Whatever I have done is due to such guidance and assistance and I would not forget them.

First and foremost I heartily thank our supervisor of this project **Mr. Narasa Reddy** for the valuable guidance and advice. He inspired us greatly to work on this project. I am extremely grateful to her for providing rigorous and guidance.

Besides, I would also like to thank our Principal **Mrs. Meetali Archit** and our beloved School Nagarjuna Vidyaniketan for providing such good environment and facilities to complete this project.

I also thank my teammates Kalidindi Ritika and Midhuna Arun for being my project partners and helping me get the expected results.

Finally, an honourable mention to our families and friends for their suggestions, support and constructive criticism that helped us in completing this project.

INDEX

SNO.	TOPIC	PAGE NO.
1.	Introduction	5
2.	Objective	6
3.	Modules Used	6
4.	System requirements	6
5.	Algorithm	7
6.	Flowchart	8
7.	How to play?	9
8.	Source Code	10
9.	Output	35
10.	Conclusion	44
11.	Bibliography	44

INTRODUCTION

Minesweeper is single-player logic-based computer game played on rectangular board whose object is to locate a predetermined number of randomly-placed "mines" in the shortest possible time by clicking on "safe" squares while avoiding the objective is to clear the board without detonating the mines i.e. avoiding the grid with a mine. If the player clicks on a mine, the game ends. Otherwise, a number between 0 and 8 is displayed that identifies the total number of mines present in the eight neighbouring squares. Therefore, finding a square containing "8" indicated that all eight adjacent squares contain mines, while if a zero (displayed as a blank) is uncovered, there are no mines in the surrounding squares. Minesweeper is similar to a Sudoku puzzle in that your success is largely contingent on being able to eliminate possible answers until only one answer remains.



OBJECTIVE OF THE PROJECT

It should be noted that a Computer Science course is generally at a higher mathematical and logical level therefore, the successful use of Minesweeper to assist in teaching logical proofs, and the importance of logic in hypothetical thinking and computer usage, suggests that learning Minesweeper may inherently help people to improve their hypothetical thinking and computer usage to create the understanding of the importance of making correct inferences and thinking beyond the given in a logical manner.

MODULES USED

Random module:

-Randint(): This method takes two parameters a,b in which first one is lower and second one is upper limit. This

may return any number between the two numbers including both limits.

SYSTEM REQUIREMENTS

1. HARDWARE:

Processor, keyboard & minimum memory - 2GB

2. SOFTWARE:

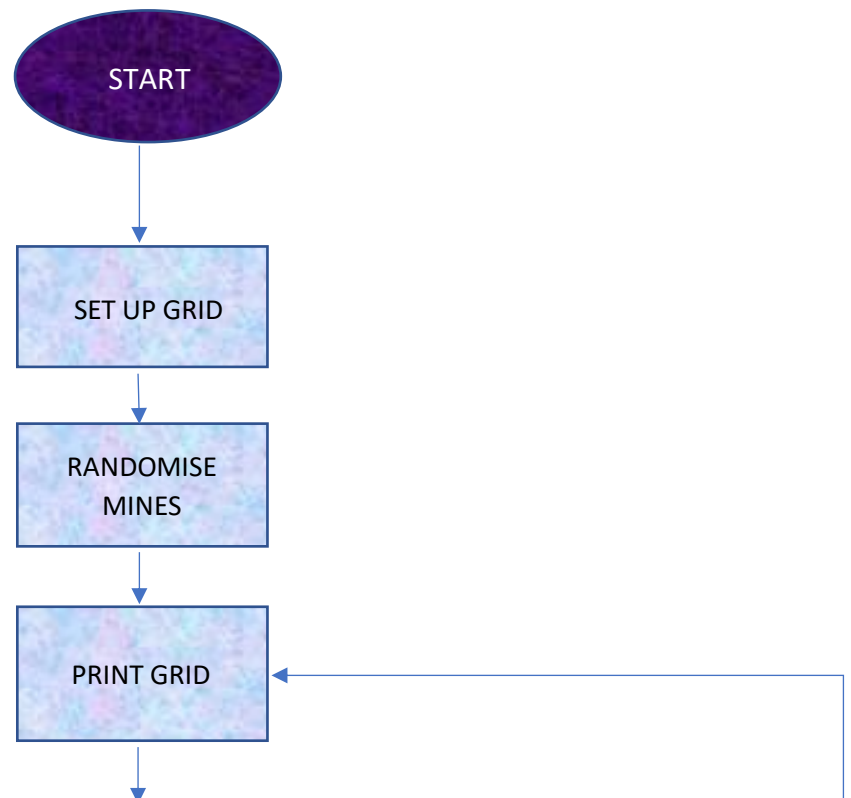
Operating System – OS7, OS10 Python IDLE

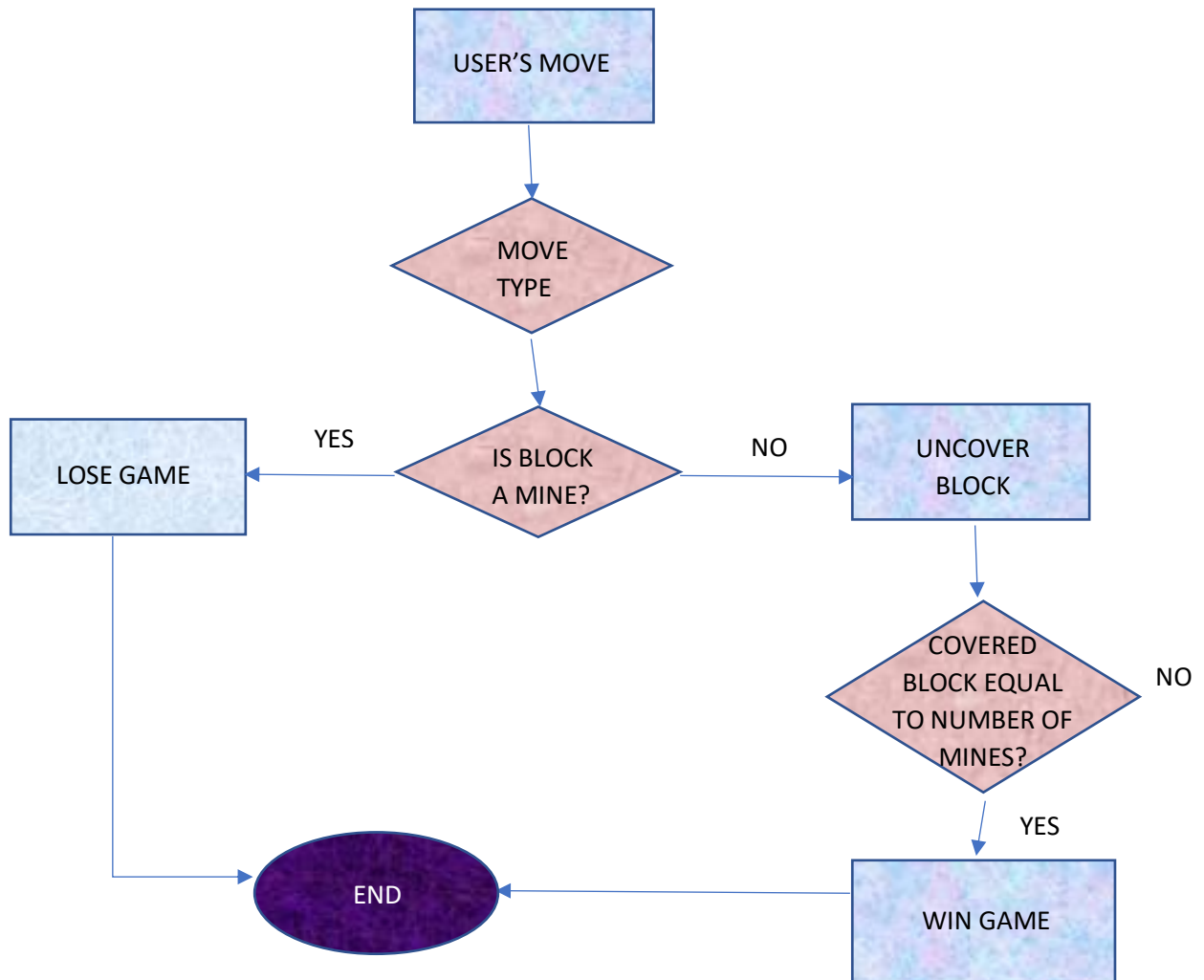
ALGORITHM

1. Start
2. Import random module.
3. Create an unopened board i.e. 20 lists having 20 elements each.
4. Merge all 20 lists in one for easier selection (nested list)
5. Asking user to select a block by entering desired row and column number.
6. Create list to store values.
7. Choosing number of bomb in each list/row using randint().
8. Create user defined function to add mine places into the list.
9. Removal of mine if it appears on first click.
10. Count mines and add numbers to the list.
11. Check if chosen block is a mine.
12. Passed if chosen block is a mine.

13. Replaced with a numeric value or blank space if the chosen block isn't a mine.
14. Convert all numeric values to string.
15. Open board according to user's choice.
16. 3x3 opening around selected block.
17. 5x5 opening around selected block if no mine in 3x3 opening.
18. Continual selection of row and column number.
19. End the execution when the selected block is a mine or the board is cleared.
20. Stop

FLOWCHART





HOW TO PLAY MINESWEEPER

1. A 20x20 grid is displayed which represents the minesweeper board.
2. Enter desired row and column number.
3. The selected grid could display a number, blank space or a mine (however, a mine will not be displayed in the first try).
 - (i) If a number is displayed, it represents the number of mines adjacent to the block.
 - (ii) If a blank space is displayed, there are no mines adjacent to the block.
 - (iii) If a mine is displayed, you lose the game.
4. Assuming that a mine hasn't been displayed, several surrounding blocks open up.

5. Enter the row and column number according to the blocks that have opened up which serve as a guidance for your next move.
6. Continue with the same until you land on a mine or clear the game.

SOURCE CODE

```
import random
```

```
D1=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D2=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D3=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D4=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D5=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
```

```

D6=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D7=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D8=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D9=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D10=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D11=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D12=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D13=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D14=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D15=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D16=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D17=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D18=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D19=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]
D20=["□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□","□"]

```

```

DALL=[D1,D2,D3,D4,D5,D6,D7,D8,D9,D10,D11,D12,D13,D14,D15,D16,D17,D18,D19,D20]

```

```

for lizt in DALL:
    print(lizt)

```

```

ch1=int(input("Enter row= "))
ch2=int(input("Enter column= "))

```

#Changing values according to indexing rules(to use in 2D list).

ch1=ch1-1

ch2=ch2-1

#Creating lists to store values.

L1=[]

L2=[]

L3=[]

L4=[]

L5=[]

L6=[]

L7=[]

L8=[]

L9=[]

L10=[]

L11=[]

L12=[]

L13=[]

L14=[]

L15=[]

L16=[]

L17=[]

L18=[]

L19=[]

L20=[]

#Randomly choosing number of bombs in each list/row.

n1=random.randint(1,3)

n2=random.randint(1,3)

n3=random.randint(1,3)

n4=random.randint(1,3)

n5=random.randint(1,3)

n6=random.randint(1,3)

n7=random.randint(1,3)

n8=random.randint(1,3)

```
n9=random.randint(1,3)
n10=random.randint(1,3)
n11=random.randint(1,3)
n12=random.randint(1,3)
n13=random.randint(1,3)
n14=random.randint(1,3)
n15=random.randint(1,3)
n16=random.randint(1,3)
n17=random.randint(1,3)
n18=random.randint(1,3)
n19=random.randint(1,3)
n20=random.randint(1,3)
```

#User-defined function to add bombs and non-bomb places into the list.

```
def randomiser(L,n):
    ran=random.randint(0,19)
    for i in range(ran):
        L.append("_")
    L.append("*")
    for i in range(19-ran):
        L.append("_")

    for j in range(n-1):
        ran2=random.randint(0,19)

        if ran2==ran:
            ran2=random.randint(0,19)
            L[ran2]="*"
        else:
            L[ran2]="*"
    randomiser(L1,n1)
    randomiser(L2,n2)
    randomiser(L3,n3)
    randomiser(L4,n4)
    randomiser(L5,n5)
    randomiser(L6,n6)
    randomiser(L7,n7)
    randomiser(L8,n8)
    randomiser(L9,n9)
```

```

randomiser(L10,n10)
randomiser(L11,n11)
randomiser(L12,n12)
randomiser(L13,n13)
randomiser(L14,n14)
randomiser(L15,n15)
randomiser(L16,n16)
randomiser(L17,n17)
randomiser(L18,n18)
randomiser(L19,n19)
randomiser(L20,n20)

```

```

ALL=[L1,L2,L3,L4,L5,L6,L7,L8,L9,L10,L11,L12,L13,L14,L15,L16,
L17,L18,L19,L20] #2D list for easier selection

```

#First click cannot be a bomb. Removing bomb if it appears on first click.

```

if ALL[ch1][ch2]=='*':
    ALL[ch1][ch2]='_'

```

#Counting bombs and adding numbers to the lists.

```

for i in range(20):
    for j in range(20):
        if ALL[i][j]=='*': #Checking if chosen block is a bomb.
            try:
                if ALL[i][j-1]=='*': #If the place next to it is a bomb,
                    pass #it is passed.
                elif ALL[i][j-1]=='_': #If the place next to it isn't a bomb,
                    ALL[i][j-1]=1 #We replace it with a numeric value to add on
to it later.
            except IndexError:
                pass
            try:
                if ALL[i][j+1]=='*':
                    pass
                elif ALL[i][j+1]=='_':
                    ALL[i][j+1]=1
            except IndexError:
                pass
            try:

```

```

    if ALL[i+1][j]=="*":
        pass
    elif ALL[i+1][j]=="_":
        ALL[i+1][j]=1
    elif ALL[i+1][j] in [1,2,3]:
        ALL[i+1][j]+=1
    elif ALL[i+1][j]==4:
        pass
except IndexError:
    pass
try:
    if ALL[i+1][j-1]=="*":
        pass
    elif ALL[i+1][j-1]=="_":
        ALL[i+1][j-1]=1
    elif ALL[i+1][j-1] in [1,2,3]:
        ALL[i+1][j-1]+=1
    elif ALL[i+1][j-1]==4:
        pass
except IndexError:
    pass
try:
    if ALL[i+1][j+1]=="*":
        pass
    elif ALL[i+1][j+1]=="_":
        ALL[i+1][j+1]=1
    elif ALL[i+1][j+1] in [1,2,3]:
        ALL[i+1][j+1]+=1
    elif ALL[i+1][j+1]==4:
        pass
except IndexError:
    pass
try:
    if ALL[i-1][j]=="*":
        pass
    elif ALL[i-1][j]=="_":
        ALL[i-1][j]=1
    elif ALL[i-1][j] in [1,2,3]:
        ALL[i-1][j]+=1

```

```

        elif ALL[i-1][j]==4:
            pass
    except IndexError:
        pass
    try:
        if ALL[i-1][j-1]=="*":
            pass
        elif ALL[i-1][j-1]=="_":
            ALL[i-1][j-1]=1
        elif ALL[i-1][j-1] in [1,2,3]:
            ALL[i-1][j-1]+=1
        elif ALL[i-1][j-1]==4:
            pass
    except IndexError:
        pass
    try:
        if ALL[i-1][j+1]=="*":
            pass
        elif ALL[i-1][j+1]=="_":
            ALL[i-1][j+1]=1
        elif ALL[i-1][j+1] in [1,2,3]:
            ALL[i-1][j+1]+=1
        elif ALL[i-1][j+1]==4:
            pass
    except IndexError:
        pass

```

```

ALL=[L1,L2,L3,L4,L5,L6,L7,L8,L9,L10,L11,L12,L13,L14,L15,L16,
L17,L18,L19,L20]

```

```

for wt in range(20):

```

```

    for wt2 in range(20):

```

```

        if ALL[wt][wt2] in [1,2,3,4]:

```

```

            ALL[wt][wt2] = str(ALL[wt][wt2]) #Converting all numeric
values to string.

```

```

#Number of mines in the board.

```

```

bcount=0

```

```

for l in ALL:

```

```

    for b in l:
        if b=='*':
            bcount+1

cleancount=400-bcount  #Number of clean boxes in the board.


#Opening the DALL board according to user's choice.
#Swapping out items of list DALL with list ALL's items.
nval=ALL[ch1][ch2]
DALL[ch1][ch2]=nval

lclean=1 #Number of clean boxes user has opened.


#3X3 opening around the chosen box.
try:
    if ch2-1<0: #To avoid negative indexing, we pass if the value becomes
negative.
        pass
    elif ALL[ch1][ch2-1] not in '*': #If the nearby boxes are not bombs,
we open it
        nval=ALL[ch1][ch2-1] #by swapping out the boxes in list DALL.
        DALL[ch1][ch2-1]=nval
        lclean+=1 #adding to lclean when a new clean box is opened.
except IndexError:
    pass

try:
    if ALL[ch1][ch2+1] not in '*':
        nval=ALL[ch1][ch2+1]
        DALL[ch1][ch2+1]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-1<0:
        pass

```

```

        elif ALL[ch1-1][ch2] not in '*':
            nval=ALL[ch1-1][ch2]
            DALL[ch1-1][ch2]=nval
            lclean+=1
except IndexError:
    pass

try:
    if ch1-1<0 or ch2-1<0:
        pass
    elif ALL[ch1-1][ch2-1] not in '*':
        nval=ALL[ch1-1][ch2-1]
        DALL[ch1-1][ch2-1]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-1<0:
        pass
    elif ALL[ch1-1][ch2+1] not in '*':
        nval=ALL[ch1-1][ch2+1]
        DALL[ch1-1][ch2+1]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+1][ch2] not in '*':
        nval=ALL[ch1+1][ch2]
        DALL[ch1+1][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch2-1<0:
        pass
    elif ALL[ch1+1][ch2-1] not in '*':

```

```

        nval=ALL[ch1+1][ch2-1]
        DALL[ch1+1][ch2-1]=nval
        lclean+=1
except IndexError:
    pass

```

```

try:
    if ALL[ch1+1][ch2+1] not in '*':
        nval=ALL[ch1+1][ch2+1]
        DALL[ch1+1][ch2+1]=nval
        lclean+=1
except IndexError:
    pass

```

#5x5 opening around the 3x3 if there no bomb in the 3x3 opening.

```

try:
    if ALL[ch1][ch2-1] not in '*' and ALL[ch1][ch2+1] not in '*' and
    ALL[ch1-1][ch2] not in '*' and ALL[ch1-1][ch2-1] not in '*' and
    ALL[ch1-1][ch2+1] not in '*' and ALL[ch1+1][ch2] not in '*' and
    ALL[ch1+1][ch2-1] not in '*' and ALL[ch1+1][ch2+1] not in '*':
        try:
            if ch2-2<0:
                pass
            elif ALL[ch1][ch2-2] not in '*':
                nval=ALL[ch1][ch2-2]
                DALL[ch1][ch2-2]=nval
                lclean+=1
        except IndexError:
            pass

        try:
            if ALL[ch1][ch2+2] not in '*':
                nval=ALL[ch1][ch2+2]
                DALL[ch1][ch2+2]=nval
                lclean+=1
        except IndexError:
            pass

```

```

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2] not in '*':
        nval=ALL[ch1-2][ch2]
        DALL[ch1-2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0 or ch2-2<0:
        pass
    elif ALL[ch1-2][ch2-2] not in '*':
        nval=ALL[ch1-2][ch2-2]
        DALL[ch1-2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2+2] not in '*':
        nval=ALL[ch1-2][ch2+2]
        DALL[ch1-2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2] not in '*':
        nval=ALL[ch1+2][ch2]
        DALL[ch1+2][ch2]=nval
        lclean+=1
except IndexError:
    pass

```

```

try:
    if ch2-2<0:
        pass
    elif ALL[ch1+2][ch2-2] not in '*':
        nval=ALL[ch1+2][ch2-2]
        DALL[ch1+2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

```

```

try:
    if ALL[ch1+2][ch2+2] not in '*':
        nval=ALL[ch1+2][ch2+2]
        DALL[ch1+2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

```

except IndexError: **#Python raises index error when the values are 1 or 20 in the list conditions**

```

    if ch1==1:
        if ALL[ch1][ch2-1] not in '*' and ALL[ch1][ch2+1] not in '*'
        and ALL[ch1+1][ch2] not in '*' and ALL[ch1+1][ch2-1] not in '*'
        and ALL[ch1+1][ch2+1] not in '*':
            try:
                if ch2-2<0:
                    pass
                elif ALL[ch1][ch2-2] not in '*':
                    nval=ALL[ch1][ch2-2]
                    DALL[ch1][ch2-2]=nval
                    lclean+=1
            except IndexError:
                pass

```

```

try:
    if ALL[ch1][ch2+2] not in '*':
        nval=ALL[ch1][ch2+2]
        DALL[ch1][ch2+2]=nval
        lclean+=1

```

```

except IndexError:
    pass

try:
    if ALL[ch1+2][ch2] not in '*':
        nval=ALL[ch1+2][ch2]
        DALL[ch1+2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch2-2<0:
        pass
    elif ALL[ch1+2][ch2-2] not in '*':
        nval=ALL[ch1+2][ch2-2]
        DALL[ch1+2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2+2] not in '*':
        nval=ALL[ch1+2][ch2+2]
        DALL[ch1+2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

if ch2==1:
    if ALL[ch1][ch2+1] not in '*' and ALL[ch1-1][ch2] not in '*'
    and ALL[ch1-1][ch2+1] not in '*' and ALL[ch1+1][ch2] not in '*'
    and ALL[ch1+1][ch2+1] not in '*':
        try:
            if ALL[ch1][ch2+2] not in '*':
                nval=ALL[ch1][ch2+2]
                DALL[ch1][ch2+2]=nval
                lclean+=1
        except IndexError:
            pass

```

```

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2] not in '*':
        nval=ALL[ch1-2][ch2]
        DALL[ch1-2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2+2] not in '*':
        nval=ALL[ch1-2][ch2+2]
        DALL[ch1-2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2] not in '*':
        nval=ALL[ch1+2][ch2]
        DALL[ch1+2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2+2] not in '*':
        nval=ALL[ch1+2][ch2+2]
        DALL[ch1+2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass
if ch1==20:

```

```
    if ALL[ch1][ch2-1] not in '*' and ALL[ch1][ch2+1] not in '*'
and ALL[ch1-1][ch2] not in '*' and ALL[ch1-1][ch2-1] not in '*' and
ALL[ch1-1][ch2+1] not in '*':
```

```
    try:
```

```
        if ch2-2<0:
```

```
            pass
```

```
        elif ALL[ch1][ch2-2] not in '*':
```

```
            nval=ALL[ch1][ch2-2]
```

```
            DALL[ch1][ch2-2]=nval
```

```
            lclean+=1
```

```
    except IndexError:
```

```
        pass
```

```
    try:
```

```
        if ALL[ch1][ch2+2] not in '*':
```

```
            nval=ALL[ch1][ch2+2]
```

```
            DALL[ch1][ch2+2]=nval
```

```
            lclean+=1
```

```
    except IndexError:
```

```
        pass
```

```
    try:
```

```
        if ch1-2<0:
```

```
            pass
```

```
        elif ALL[ch1-2][ch2] not in '*':
```

```
            nval=ALL[ch1-2][ch2]
```

```
            DALL[ch1-2][ch2]=nval
```

```
            lclean+=1
```

```
    except IndexError:
```

```
        pass
```

```
    try:
```

```
        if ch1-2<0 or ch2-2<0:
```

```
            pass
```

```
        elif ALL[ch1-2][ch2-2] not in '*':
```

```
            nval=ALL[ch1-2][ch2-2]
```

```
            DALL[ch1-2][ch2-2]=nval
```

```
            lclean+=1
```

```
    except IndexError:
```

```

pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2+2] not in '*':
        nval=ALL[ch1-2][ch2+2]
        DALL[ch1-2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

if ch2==20:
    if ALL[ch1][ch2-1] not in '*' and ALL[ch1-1][ch2] not in '*'
and ALL[ch1-1][ch2-1] not in '*' and ALL[ch1+1][ch2] not in '*'
and ALL[ch1+1][ch2-1] not in '*':
        try:
            if ch2-2<0:
                pass
            elif ALL[ch1][ch2-2] not in '*':
                nval=ALL[ch1][ch2-2]
                DALL[ch1][ch2-2]=nval
                lclean+=1
        except IndexError:
            pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2] not in '*':
        nval=ALL[ch1-2][ch2]
        DALL[ch1-2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0 or ch2-2<0:
        pass
    elif ALL[ch1-2][ch2-2] not in '*':

```

```

        nval=ALL[ch1-2][ch2-2]
        DALL[ch1-2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2] not in '*':
        nval=ALL[ch1+2][ch2]
        DALL[ch1+2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch2-2<0:
        pass
    elif ALL[ch1+2][ch2-2] not in '*':
        nval=ALL[ch1+2][ch2-2]
        DALL[ch1+2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

print("-----")
for lizt in DALL:
    print(lizt)

#-----
#Repeating until user clicks on a bomb.

while True:
    ch1=int(input("Enter row= ")) #Repetitive choosing of boxes.
    ch2=int(input("Enter column= "))
    ch1=ch1-1
    ch2=ch2-1
    if ALL[ch1][ch2]=='*': #if the chosen box is a bomb,
        for lizt in ALL:
            print(lizt)

```

```

    print("Oops! You stepped on a mine :(")
    break #the while loop is broken.
else:      #if not, it continues until the user clicks on a bomb.
    nval=ALL[ch1][ch2]
    DALL[ch1][ch2]=nval
    lclean+=1
try:
    if ch2-1<0:
        pass

    elif ALL[ch1][ch2-1] not in '*':
        nval=ALL[ch1][ch2-1]
        DALL[ch1][ch2-1]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1][ch2+1] not in '*':
        nval=ALL[ch1][ch2+1]
        DALL[ch1][ch2+1]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-1<0:
        pass
    elif ALL[ch1-1][ch2] not in '*':
        nval=ALL[ch1-1][ch2]
        DALL[ch1-1][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-1<0 or ch2-1<0:
        pass
    elif ALL[ch1-1][ch2-1] not in '*':

```

```

        nval=ALL[ch1-1][ch2-1]
        DALL[ch1-1][ch2-1]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-1<0:
        pass
    elif ALL[ch1-1][ch2+1] not in '*':
        nval=ALL[ch1-1][ch2+1]
        DALL[ch1-1][ch2+1]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+1][ch2] not in '*':
        nval=ALL[ch1+1][ch2]
        DALL[ch1+1][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch2-1<0:
        pass
    elif ALL[ch1+1][ch2-1] not in '*':
        nval=ALL[ch1+1][ch2-1]
        DALL[ch1+1][ch2-1]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+1][ch2+1] not in '*':
        nval=ALL[ch1+1][ch2+1]
        DALL[ch1+1][ch2+1]=nval

```

```

        lclean+=1
except IndexError:
    pass
try:
    if ALL[ch1][ch2-1] not in '*' and ALL[ch1][ch2+1] not in '*'
and ALL[ch1-1][ch2] not in '*' and ALL[ch1-1][ch2-1] not in '*' and
ALL[ch1-1][ch2+1] not in '*' and ALL[ch1+1][ch2] not in '*' and
ALL[ch1+1][ch2-1] not in '*' and ALL[ch1+1][ch2+1] not in '*':
        try:
            if ch2-2<0:
                pass
            elif ALL[ch1][ch2-2] not in '*':
                nval=ALL[ch1][ch2-2]
                DALL[ch1][ch2-2]=nval
                lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1][ch2+2] not in '*':
        nval=ALL[ch1][ch2+2]
        DALL[ch1][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2] not in '*':
        nval=ALL[ch1-2][ch2]
        DALL[ch1-2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0 or ch2-2<0:
        pass

```

```

        elif ALL[ch1-2][ch2-2] not in '*':
            nval=ALL[ch1-2][ch2-2]
            DALL[ch1-2][ch2-2]=nval
            lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2+2] not in '*':
        nval=ALL[ch1-2][ch2+2]
        DALL[ch1-2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2] not in '*':
        nval=ALL[ch1+2][ch2]
        DALL[ch1+2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch2-2<0:
        pass
    elif ALL[ch1+2][ch2-2] not in '*':
        nval=ALL[ch1+2][ch2-2]
        DALL[ch1+2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2+2] not in '*':
        nval=ALL[ch1+2][ch2+2]

```

```

        DALL[ch1+2][ch2+2]=nval
        lclean+=1
    except IndexError:
        pass

```

#When if statment's conditions are out of indexing range:

```

    except IndexError:
        if ch1==1:
            if ALL[ch1][ch2-1] not in '*' and ALL[ch1][ch2+1] not in '*'
            and ALL[ch1+1][ch2] not in '*' and ALL[ch1+1][ch2-1] not in '*'
            and ALL[ch1+1][ch2+1] not in '*':

```

```

                try:
                    if ch2-2<0:
                        pass
                    elif ALL[ch1][ch2-2] not in '*':
                        nval=ALL[ch1][ch2-2]
                        DALL[ch1][ch2-2]=nval
                        lclean+=1

```

```

    except IndexError:
        pass

```

```

    try:
        if ALL[ch1][ch2+2] not in '*':
            nval=ALL[ch1][ch2+2]
            DALL[ch1][ch2+2]=nval
            lclean+=1

```

```

    except IndexError:
        pass

```

```

    try:
        if ALL[ch1+2][ch2] not in '*':
            nval=ALL[ch1+2][ch2]
            DALL[ch1+2][ch2]=nval
            lclean+=1

```

```

    except IndexError:
        pass

```

```

    try:
        if ch2-2<0:

```

```

        pass
    elif ALL[ch1+2][ch2-2] not in '*':
        nval=ALL[ch1+2][ch2-2]
        DALL[ch1+2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2+2] not in '*':
        nval=ALL[ch1+2][ch2+2]
        DALL[ch1+2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass
if ch2==1:
    if ALL[ch1][ch2+1] not in '*' and ALL[ch1-1][ch2] not in '*'
and ALL[ch1-1][ch2+1] not in '*' and ALL[ch1+1][ch2] not in '*'
and ALL[ch1+1][ch2+1] not in '*':
        try:
            if ALL[ch1][ch2+2] not in '*':
                nval=ALL[ch1][ch2+2]
                DALL[ch1][ch2+2]=nval
                lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2] not in '*':
        nval=ALL[ch1-2][ch2]
        DALL[ch1-2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:

```

```

        pass
    elif ALL[ch1-2][ch2+2] not in '*':
        nval=ALL[ch1-2][ch2+2]
        DALL[ch1-2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2] not in '*':
        nval=ALL[ch1+2][ch2]
        DALL[ch1+2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2+2] not in '*':
        nval=ALL[ch1+2][ch2+2]
        DALL[ch1+2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

if ch1==20:
    if ALL[ch1][ch2-1] not in '*' and ALL[ch1][ch2+1] not in '*'
    and ALL[ch1-1][ch2] not in '*' and ALL[ch1-1][ch2-1] not in '*' and
    ALL[ch1-1][ch2+1] not in '*':
        try:
            if ch2-2<0:
                pass
            elif ALL[ch1][ch2-2] not in '*':
                nval=ALL[ch1][ch2-2]
                DALL[ch1][ch2-2]=nval
                lclean+=1
        except IndexError:
            pass

try:
    if ALL[ch1][ch2+2] not in '*':

```

```

        nval=ALL[ch1][ch2+2]
        DALL[ch1][ch2+2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2] not in '*':
        nval=ALL[ch1-2][ch2]
        DALL[ch1-2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0 or ch2-2<0:
        pass
    elif ALL[ch1-2][ch2-2] not in '*':
        nval=ALL[ch1-2][ch2-2]
        DALL[ch1-2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2+2] not in '*':
        nval=ALL[ch1-2][ch2+2]
        DALL[ch1-2][ch2+2]=nval
        lclean+=1
except IndexError:
    pass
if ch2==20:
    if ALL[ch1][ch2-1] not in '*' and ALL[ch1-1][ch2] not in '*'
and ALL[ch1-1][ch2-1] not in '*' and ALL[ch1+1][ch2] not in '*'
and ALL[ch1+1][ch2-1] not in '*':

```

```

try:
    if ch2-2<0:
        pass
    elif ALL[ch1][ch2-2] not in '*':
        nval=ALL[ch1][ch2-2]
        DALL[ch1][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0:
        pass
    elif ALL[ch1-2][ch2] not in '*':
        nval=ALL[ch1-2][ch2]
        DALL[ch1-2][ch2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ch1-2<0 or ch2-2<0:
        pass
    elif ALL[ch1-2][ch2-2] not in '*':
        nval=ALL[ch1-2][ch2-2]
        DALL[ch1-2][ch2-2]=nval
        lclean+=1
except IndexError:
    pass

try:
    if ALL[ch1+2][ch2] not in '*':
        nval=ALL[ch1+2][ch2]
        DALL[ch1+2][ch2]=nval
        lclean+=1
except IndexError:
    pass

```

```

try:

```

```

        if ch2-2<0:
            pass
        elif ALL[ch1+2][ch2-2] not in '*':
            nval=ALL[ch1+2][ch2-2]
            DALL[ch1+2][ch2-2]=nval
            lclean+=1
    except IndexError:
        pass

for lizt in DALL:
    print(lizt)
    if lclean==cleancount: #if the number of clean boxes chosen by the
        user is equal to the number of clean boxes,
        print("CONGRATS! You've won!") #the user has won the game.
        break

```

OUTPUT

[illegible]

Enter row= 5

Enter column= 4

[illegible]

Enter row= 3

Enter column= 9

[illegible]

Enter row= 6

Enter column= 2

[illegible]

Enter row= 7

Enter column= 8

[illegible]

Enter row= 5

Enter column= 6

[illegible]

Enter row= 9

Enter column= 1

[illegible]

Enter row= 9

Enter column= 9

[illegible]

Enter row= 7

Enter column= 7

```
['1', '*', '1', '1', '*', '2', '1', '2', '1', '1', '_', '1', '1', '1', '1', '*', '1', '_', '_', '_']
['1', '1', '1', '1', '1', '1', '*', '1', '*', '1', '_', '1', '*', '1', '1', '1', '1', '_', '1', '1']
['_', '_', '_', '_', '_', '2', '2', '3', '1', '1', '_', '1', '1', '1', '1', '_', '_', '_', '1', '*']
['_', '_', '_', '_', '_', '1', '*', '2', '1', '_', '_', '_', '_', '_', '_', '_', '_', '1', '1']
['_', '_', '_', '1', '1', '2', '1', '*', '1', '_', '_', '_', '_', '_', '_', '_', '_', '_', '_']
['1', '1', '1', '1', '1', '2', '2', '2', '1', '_', '_', '_', '_', '_', '_', '_', '_', '_', '_']
['2', '*', '2', '1', '1', '1', '1', '1', '_', '_', '_', '_', '_', '_', '_', '_', '_', '_']
['1', '*', '1', '_', '_', '1', '2', '2', '1', '_', '_', '_', '_', '_', '_', '_', '_', '_', '_']
['1', '1', '1', '_', '_', '_', '2', '*', '2', '_', '_', '_', '_', '_', '_', '_', '_', '_', '_']
['_', '_', '1', '1', '1', '_', '1', '*', '1', '_', '_', '_', '_', '_', '_', '_', '_', '1', '1', '1', '_']
['_', '1', '2', '*', '1', '_', '1', '2', '2', '1', '_', '_', '_', '_', '_', '_', '1', '*', '1', '_']
['_', '1', '*', '2', '2', '1', '_', '1', '*', '1', '_', '_', '_', '_', '_', '_', '1', '1', '1', '_']
['_', '1', '2', '2', '*', '1', '_', '1', '1', '2', '1', '1', '_', '_', '_', '_', '_', '_', '_']
['_', '1', '2', '*', '1', '2', '1', '1', '1', '2', '*', '1', '_', '_', '_', '_', '_', '_', '_']
['_', '1', '*', '2', '2', '2', '*', '1', '1', '*', '1', '1', '_', '_', '_', '_', '_', '_', '_']
['_', '2', '2', '2', '*', '1', '1', '1', '1', '1', '1', '1', '_', '_', '_', '_', '_', '_', '_']
['_', '1', '*', '2', '2', '2', '_', '_', '_', '_', '_', '_', '_', '_', '1', '1', '2', '1', '1']
['_', '1', '1', '1', '1', '*', '1', '_', '1', '1', '1', '1', '_', '_', '_', '_', '1', '*', '1', '*', '1']
['_', '_', '_', '1', '1', '1', '_', '1', '*', '2', '1', '_', '_', '1', '1', '2', '1', '2', '2', '2']
['1', '1', '1', '1', '1', '1', '_', '1', '1', '*', '1', '_', '_', '1', '*', '1', '1', '_', '1', '*']
```

Oops! You stepped on a mine :(

CONCLUSION

This project, “Minesweeper” has a lot of potential to grow by enhancing the existing base with improvised and advanced features.

Efficient adjuncts like a timer for challenging the user with their time management skills along with proficient thinking would unlock the game’s capability.

I appreciate each and every person who helped me in this project and would like to improve this with them.

BIBLIOGRAPHY

1. Computer Science NCERT textbook- 11
2. Computer Science NCERT textbook- 12
3. <https://pythontutor.com>